



Tecnun
Universidad
de Navarra

ESCUELA DE INGENIERÍA
INGENIARITZA ESKOLA
SCHOOL OF ENGINEERING

Computer Science II

Input & output data in C++

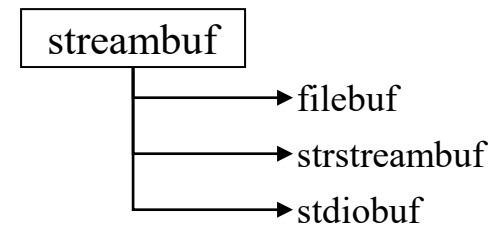
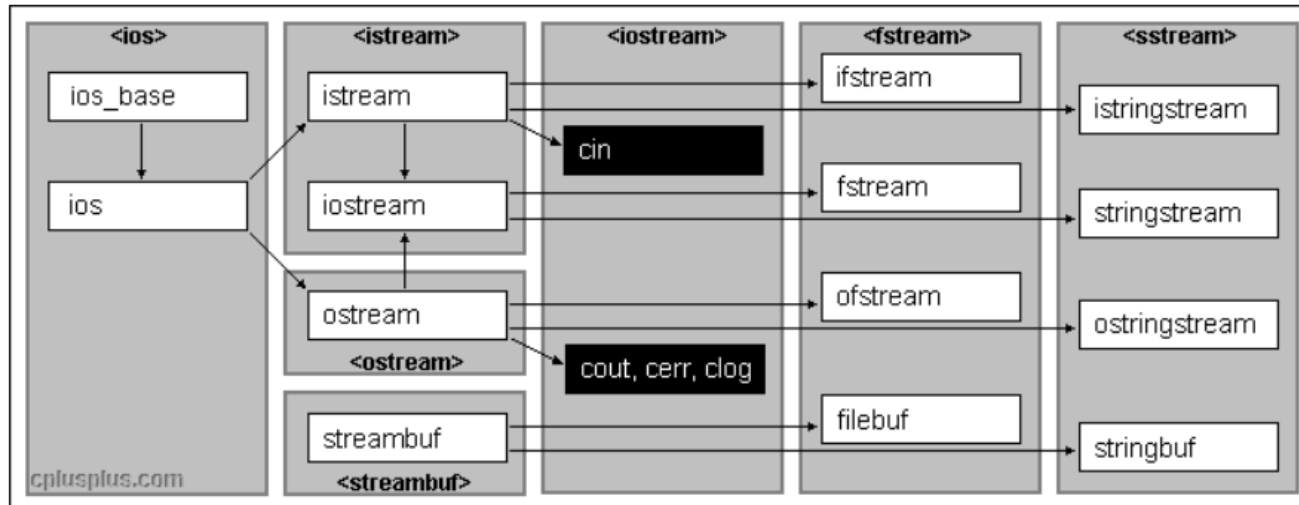
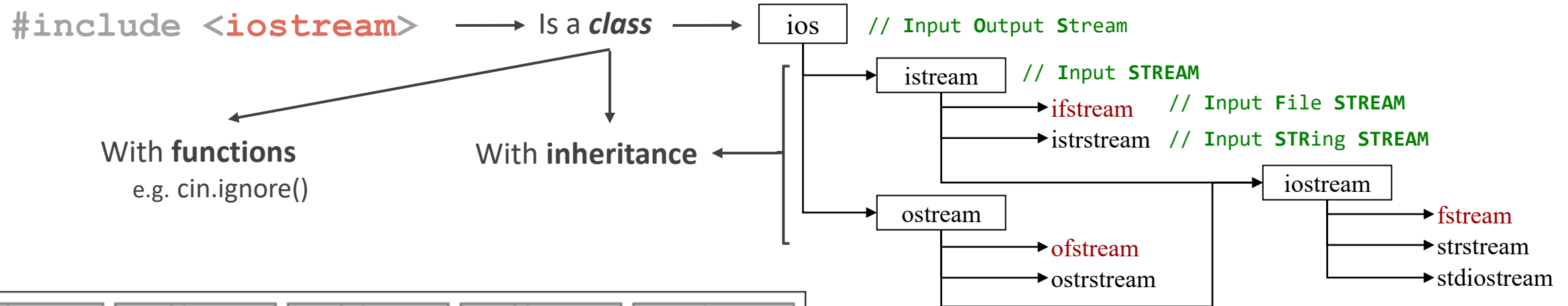
Ph.D. Paul Bustamante y Ph.D. Jesús Paredes

Índice



- 1. Input/output data in C++**
- 2. Input/output with format**
- 3. Enable/Disable indicators**
- 4. Input/output manipulator**
- 5. Functions width(), precision() y fill()**
- 6. Input/output files**

Input/output data in C++





Input/output data in C++

Based on “**classes**” and “**hiararchy**” (easy to extend and modify).

An opened file is represented by a **Stream (flow)**: produces or consumes information.

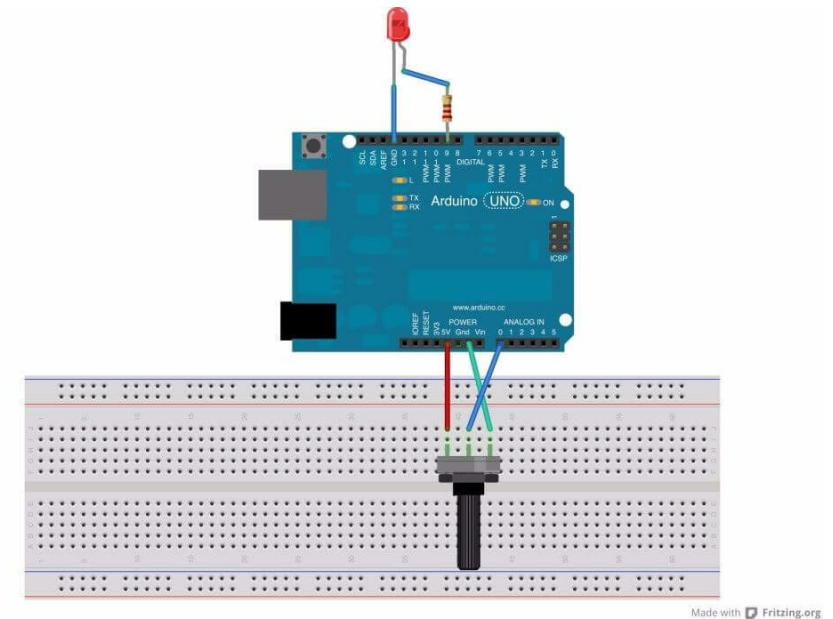
- A **stream** is always linked to a physical device.
- All the **streams** have a similar behavior.
- Open **streams** in any program

OBJECTS

◦ cin:	standard input (keyboard)	// Call-INput
◦ cout:	standard output (screen)	// Call-OUTput
◦ cerr:	error message output (screen)	// Call-ERRor

There are two class hierarchies

- Low level: **streambuf** (advanced users)
- High level: **istream**, **ostream** y **iostream**, derived from **ios**
- These classes have variables and methods to control input and output **streams**.



Made with Fritzing.org



Input/output with format

Format flag manipulators (functions):

Independent flags (switch on):

boolalpha	Alphanumerical bool values
showbase	Show numerical base prefixes
showpoint	Show decimal point
showpos	Show positive signs
skipws	Skip whitespaces
unitbuf	Flush buffer after insertions
uppercase	Generate upper-case letters

Independent flags (switch off):

noboolalpha	No alphanumerical bool values
noshowbase	Do not show numerical base prefixes
noshowpoint	Do not show decimal point
noshowpos	Do not show positive signs
noskipws	Do not skip whitespaces
nounitbuf	Do not force flushes after insertions
nouppercase	Do not generate upper case letters

Numerical base format flags ("basefield" flags):

dec	Use decimal base
hex	Use hexadecimal base
oct	Use octal base
Floating-point format flags ("floatfield" flags):	
fixed	Use fixed floating-point notation
scientific	Use scientific floating-point notation

Adjustment format flags ("adjustfield" flags):

internal	Adjust field by inserting characters at an internal position (function)
left	Adjust output to the left (function)
right	Adjust output to the right (function)



Enable/Disable indicators

Format flag manipulators (functions):

Independent flags (switch on):

boolalpha	Alphanumerical bool values
showbase	Show numerical base prefixes
showpoint	Show decimal point
showpos	Show positive signs
skipws	Skip whitespaces
unitbuf	Flush buffer after insertions
uppercase	Generate upper-case letters

Independent flags (switch off):

noboolalpha	No alphanumerical bool values
noshowbase	Do not show numerical base prefixes
noshowpoint	Do not show decimal point
noshowpos	Do not show positive signs
noskipws	Do not skip whitespaces
nounitbuf	Do not force flushes after insertions
nouppercase	Do not generate upper case letters

```
// Ejemplo manipuladores
#include <iostream>
using namespace std;

void main()
{
    double x = 5;
    cout << "Numero: " << x;
    cout << "\nActivamos decimales (showpoint): " << showpoint << x;
    cout << "\nActivamos signo (showpos): " << showpos << x;
    cout << "\nDesactivamos decimales (noshowpoint): " << noshowpoint << x;
    cout << "\nDesactivamos signo (noshowpos):" << noshowpos << x;

    // Ejemplo manipuladores concatenados
    cout << "\nActivamos decimales y signo: " << showpoint << showpos << x;
    cout << endl;
}
```

```
Numero: 5
Activamos decimales (showpoint): 5.00000
Activamos signo (showpos): +5.00000
Desactivamos decimales (noshowpoint): +5
Desactivamos signo (noshowpos):5
Activamos decimales y signo : +5.00000
```

Enable/Disable indicators

Indicator activation: using **setf()** from **ios**:

- Its structure is as follows:

```
long setf(long indic);
```

The return value is the previous configuration; **indic** is the **long** containing the flags that you want to set.

- Multiple indicators are allowed to be activated at the same time with the binary OR operator. Example:

```
cout.setf(ios::showpoint | ios::fixed)
```

- it is necessary to determine the affected stream (**cout**) and the type of the indicator (**ios**)

To disable the flags, the **unsetf()** function is used in a similar way to **setf()**.

The function **flags()** returns a **long** with the configuration of all flags:

- Its structure is as follows:

```
long flags();
```

- There is another definition that allows changing all indicators at the same time by taking a **long** as an argument

```
long flags(long indic);
```

the return value is a **long** with previous configuration

The function **setf()** changes only the indicators passed as arguments, while **flags()** changes the entire configuration.



Enable/Disable indicators

Format flag manipulators (functions):

Independent flags (switch on):

boolalpha	Alphanumerical bool values
showbase	Show numerical base prefixes
showpoint	Show decimal point
showpos	Show positive signs
skipws	Skip whitespaces
unitbuf	Flush buffer after insertions
uppercase	Generate upper-case letters

Independent flags (switch off):

noboolalpha	No alphanumerical bool values
noshowbase	Do not show numerical base prefixes
noshowpoint	Do not show decimal point
noshowpos	Do not show positive signs
noskipws	Do not skip whitespaces
nounitbuf	Do not force flushes after insertions
nouppercase	Do not generate upper case letters

```
// Ejemplo manipuladores
#include <iostream>
using namespace std;

void main()
{
    double x = 5;
    // Funciones setf y unsetf (es lo mismo)
    cout.setf(ios::showpos | ios::showpoint);
    cout << "\nActivamos decimales y signo : " << x;
    cout.unsetf(ios::showpos | ios::showpoint);
    cout << "\nDesactivamos decimales y signo : " << x;
    cout << endl;
}
```

```
Activamos decimales y signo : +5.00000
Desactivamos decimales y signo : 5
```



Enable/Disable indicators

```
// Ejemplo manipuladores
#include <iostream>
using namespace std;

void main()
{
    double x = 5;
    // Flags (es lo mismo)
    cout << "Number 5: " << x << endl;
    a1 = cout.flags();
    cout << "Configuracion inicial A1: " << a1 << endl << endl;

    cout << showpos << "Number 5: " << x << endl;
    a2 = cout.flags();
    cout << "Configuracion actual A2: " << a2 << endl << endl;

    cout << showpoint;
    a3 = cout.flags();
    cout << "Number 5: " << x << endl;
    cout << "Configuracion actual A3: " << a3 << endl << endl;

    cout.flags(a2);
    cout << "Activar A2: " << x << endl << endl;
    cout.unsetf(a2);
    cout << "Desactivar A2: " << x << endl << endl;
}
```

```
Number 5: 5
Configuracion inicial A1: 513

Number 5: +5
Configuracion actual A2: +545

Number 5: +5.00000
Configuracion actual A3: +561

Activar A2: +5

Desactivar A2: 5
```



Function *width()*, *precision()* and *fill()*

Member function ***width()***: sets the minimum width of output data

- Its structure is as follows: `int width(int n);`
the return value is the above width.
- The set width is the minimum width. Whenever necessary, the system automatically increases it.

Member function ***precision()***: sets the number of digits for an output data:

- Its structure is as follows: `int precision(int n);`
the return value is the precision above.
- Default is 6 digits.

Member function ***fill()***: sets the fill character for output data:

- Its structure is as follows: `char fill(char ch);`
the return value is the previous fill character.
- Default is a blank space ' '.

The effect ***precision()*** and ***fill()*** is permanent. ***width()*** must be called for each data.

Example 1

```
#include <iostream>
using namespace std;

void main(){

    double coef[]={5198.0,3.21,46.32,506.5};
    char *prov[]
    ={"Madrid","Guipuzcoa","Cantabria","Alava"};

    cout.setf(ios::fixed|ios::left);//salida, alineados izq.
    cout.fill('.');//caracter para rellenar espacios

    for (int i=0;i<4;i++){
        cout.width(15);//ancho para texto
        cout << prov[i];//escribe texto
        cout.unsetf( ios::left);//suprime just. izq.
        cout.width(10);//ancho para cantidades
        cout.precision(2);//2 decimales
        cout << coef[i] << endl;
        cout.setf(ios::left);//salida, alineados a la izquierda
    }
}
```



Function *width()*, *precision()* and *fill()*

Example 1

```
#include <iostream>
using namespace std;

void main()
{
    double coef[] = { 5198.0,3.21,46.32,506.5 };
    char* prov[] = { (char*)"Madrid",(char*)"Guipuzcoa",(char*)"Cantabria",(char*)"Alava" };

    cout.setf(ios::fixed | ios::left); //salida, alineados izq.
    cout.fill('.'); //caracter para rellenar espacios

    for (int i = 0; i < 4; i++)
    {
        cout.width(15); //ancho para texto
        cout << prov[i]; //escribe texto
        cout.unsetf(ios::left); //suprime just. izq.
        cout.width(10); //ancho para cantidades
        cout.precision(2); //2 decimales
        cout << coef[i] << endl;
        cout.setf(ios::left); //salida, alineados a la izquierda
    }
}
```

```
Madrid.....5198.00
Guipuzcoa.....3.21
Cantabria.....46.32
Alava.....506.50
```



Function *width()*, *precision()* and *fill()*

Example 1

```
#include <iostream>
using namespace std;

void main()
{
    double coef[] = { 5198.0,3.21,46.32,506.5 };
    char* prov[] = { (char*)"Madrid", (char*)"Guipuzcoa", (char*)"Cantabria", (char*)"Alava" };

    cout.setf(ios::fixed | ios::left); //salida, alineados izq.
    cout.fill(' ');
    for (int i = 0; i < 4; i++) {
        cout.width(15); //ancho para texto
        cout << prov[i]; //escribe texto
        cout.unsetf(ios::left); //suprime just. izq.
        cout.width(10); //ancho para cantidades
        cout.precision(0); //4 decimales
        cout << coef[i] << endl;
        cout.setf(ios::left); //salida, alineados a la izquierda
    }
}
```

Madrid	5198
Guipuzcoa	3
Cantabria	46
Alava	507



Input/output manipulator

Manipulators are an alternative to **indicators**. They can be inserted in the input or output statement itself.

- Manipulators can have arguments or not. If they do, the **iomanip library must be included**.
- It only affects the stream (**cin**, **cout**, etc.) to which it is applied.

The manipulator **setiosflag()** is equivalent to **setf()**, and **unsetiosflag()** is equivalent to **unsetf()**.

Some input/output manipulators

- **dec**, **hex** y **oct**: set the base for integers
- **ws**: skip the initial blank spaces
- **endl**: print a '\n' and flushes the output buffer
- **flush**: flushes the output buffer
- **setw(int w)**: sets the minimum field width (only for the following data)
- **setprecision(int p)**: sets the number of digits
- **setfill(char ch)**: sets the filler character

Examples of manipulators:

```
cout << hex << 100;  
cout << setw(10) << mat[i][j] << endl;
```

Example 2

The effect of the manipulators remains in the corresponding stream until they are changed with another manipulator (except **setw()**).



Ostream Example

```
#include "prueba.h"

void main()
{
    prueba p1, p2(5,6);
    p1.pedir_Datos();
    cout << endl;
    p1.imprime_Datos();
    cout << endl;
    cout << p2;
}
```

```
x: 3
y: 2
```

```
x: 3
y: 2
```

```
x: 5
y: 6
```

```
class prueba
{
    int x;
    int y;
public:
    prueba(int _x = 0, int _y = 0) { x = _x; y = _y; }
    ~prueba() {}
    void pedir_Datos() {
        cout << "x: "; cin >> x;
        cout << "y: "; cin >> y;
    }
    void imprime_Datos(){
        cout << "x: " << x << endl;
        cout << "y: " << y << endl;
    }
    friend ostream& operator<<(ostream& objeto_os, prueba& c)
    {
        objeto_os << "x: " << c.x << endl;
        objeto_os << "y: " << c.y << endl;
        return objeto_os;
    }
};
```



Input/output data in C++

File classification:

Access:

- ~~Random Access~~
- Sequential Access

Tipos:

- Text files
- ~~Binary files~~

Opening modes:

- Writting
- Reading
- Adding

`<iostream>`

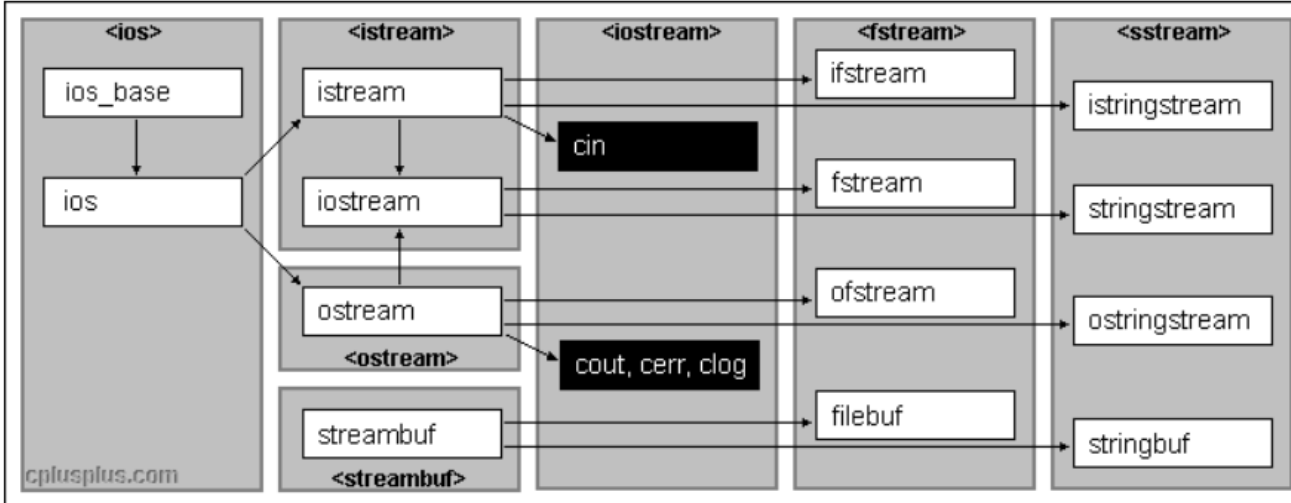
Input/Output to console

Using objects cin, cout...

`<fstream>`

Input/Output to file

Input/output files



fstream inherits all members

```
#include <iostream>
using namespace std;
```

```
void main()
{
    int x = 0;
    cout << "x? ";
    cin >> x;
}
```

```
#include <fstream>
using namespace std;

void main()
{
    int x = 0;
    fstream objeto_salida("Fichero_salida.txt", ios::out);
    objeto_salida << x;
    ofstream objeto_salida2("Fichero_salida2.txt");
    objeto_salida2 << x;
    objeto_salida.close();
    objeto_salida2.close();
}
```



Input/output files

Files are used for reading and/or writing data to permanent storage drives such as hard disks.

The required classes for using files are *ifstream*, *ofstream* and *fstream*, which are derived from *istream* and *ostream*, which are derived from the *ios* class (see initial figure).

To use them **<fstream.h>** must be included

Before opening a file, you must create a **flow** or **stream**, i.e. an object of the *ifstream*, *ofstream* or *fstream* classes, and indicate:

- Opening mode:
 - Reading, writing, adding.
- Format:
 - Text or binary.

To **write** to the file, the insertion operator "<<" overloaded is used. To **read** from the file, the extraction operator ">>" is used. This form of Write and/or Read is only in text format.

Input/output files

Class ofstream:

- Is a derived class from ***ostream***, dedicated to manipulate files on the disk ***open for writing***
- When an object of this class is constructed, the constructor automatically connects it to a ***filebuf object*** (a buffer).
- The functionality of this class is supported by the following member functions, among others:
 - **ofstream**(**const char** *nombre_fichero, **int** modo=**ios::out**, **int** proteccion=**filebuf::openprot**);
 - **void open**(**const char** *nombre_fichero, **int** modo=**ios::out**, **int** proteccion=**filebuf::openprot**);
 - **void close**(); this function closes the file
 - **bool is_open**(); checks if file is opened(=1). If it is not, it returns a 0.

```
char file[] = "Example.txt";
ofstream fout(file);
if (!fout) cerr << "It is not possible to open"<<file;
cout << "File " << file << " is opened";
fout.close();
```

```
char file[] = "Example.txt";
ofstream fout;
fout.open(file);
if (!fout) cerr << "It is not possible to open"<<file;
cout << "File " << file << " is opened? (1=true, 0=false)"<< fout.is_open();
fout << "Hola";
fout.close();
```

Input/output files

Class ifstream:

- Is a derived class from *istream*, dedicated to manipulate files on the disk ***open for reading***.
- When an object of this class is constructed, the constructor automatically connects it to a ***filebuf object*** (a buffer).
- The functionality of this class is supported by the following member functions, among others:
 - **ifstream(const char *nombre_fichero, int modo=ios::in, int proteccion=filebuf::openprot);**
 - **void open(const char *nombre_fichero, int modo=ios::in, int proteccion=filebuf::openprot);**
 - **void close();** this function closes the file
 - **bool is_open();** checks if file is opened(=1). If it is not, it returns a 0.

```
char file[] = "File_example.txt";
ofstream file1(file);
cout << endl << "File: " << file << ", its opened." << endl << endl;

file1 << "Hi. " << endl;
file1 << " I say goodbye to everyone." << endl;
file1 << "Bye. " << endl;
int x = 20;
file1 << " We convert the number 20 to hexadecimal: " << hex << x <<
endl;
file1.close();
```

```
char tmp[120];
char file[] = "File_example.txt";
ifstream file2(file);

cout << endl << "Lets read from : " << file << endl;
while (!file2.eof()) {
    file2 >> tmp; // Lee una cadena de caracteres
    cout << tmp << " "; //Muestra lo leído por consola
}
file2.close();
```



Input/output files

Class fstream:

- Is a derived class from *iostream*, dedicated to manipulate files on the disk ***open for reading and/or writing***.
- When an object of this class is constructed, the constructor automatically connects it to a ***filebuf object*** (a buffer).
- The functionality of this class is supported by the following member functions, among others :
 - `fstream( const char *nombre_fichero, int modo, int proteccion=filebuf::openprot);`
 - `void open(const char *nombre_fichero, int modo, int proteccion=filebuf::openprot);`
 - `void close();` //this function closes the file
 - `int is_open();` //checks if file is opened(=1). If it is not, it returns a 0.
- ***Opening modes:***
 - `ios::in` Open file for reading
 - `ios::out` Open file for writting
 - `ios::Nocreate` If the file does not exist, do not create it
 - `ios::app` Open the file to add more things at the end of the file (writing).

The different modes can be combined with the "|" operator, e.g. : `ios::in | ios::_Nocreate`.

Input/output files

Clase fstream (Example):

```
//generata data and write in text format
double PI = 3.141592, val;
fstream fout("datos.txt", ios::out);
if (!fout) { cerr << "Error"; return; }
for (double i = 0; i < 2 * PI; i += PI / 10)//20 ptos
{
    val = 5.0 * sin(i);
    fout << val << endl;//write into file
}
fout.close();
cout << "Data written. Now reading.." << endl;

//leer datos
fstream fin("datos.txt", ios::in | ios::_Nocreate);
if (!fin) { cerr << "Error"; return; }
cout.setf(ios::fixed | ios::showpos);//with decimal point
cout.precision(3);
while (!fin.eof()) {
    fin >> val;
    cout << val << endl;
}
fin.close();
```

```
char file[] = "File_example.txt";
fstream file1(file, ios::out);
int x = 80508;
file1 << x; // Escribe un entero
file1.close();

file1.open(file, ios::in);
char c;
cout << endl << "Lets read from : " << file << endl;
while (!file1.eof()) {
    file1 >> c;// Lee una un caracter
    cout << c << endl; //Muestra lo leído por pantalla
}
file1.close();

cout << endl;
file1.open(file);
char tmp[120];
while (!file1.eof()) {
    file1 >> tmp;// Lee una cadena de caracteres
    cout << tmp << endl; //Muestra lo leído por pantalla
}
file1.close();
```

Input/output files



```
#pragma once
#include <iostream>
#include <fstream>
using namespace std;

class prueba
{
    int x;
    int y;
public:
    prueba(int _x = 0, int _y = 0,) {
        x = _x;
        y = _y;
    }
    ~prueba() {}
    void prt(){
        cout << "x: " << x << "y: " << y << endl;
    }
    void Grabar(ofstream& ofs)
    {
        ofs << x << "," << y << endl;
    }
    void Leer(ifstream& ifs)
    {
        char ch;
        ifs >> x >> ch >> y;
    }
};
```

```
#include "prueba.h"

void main()
{
    prueba p1(5,5), p2(4,3);
    ofstream fout;
    fout.open("Datos.txt");
    p1.Grabar(fout);
    p2.Grabar(fout);
    fout.close();

    p1.prt();
    cout << endl;
    p2.prt();
    cout << endl;

    ifstream fin;
    fin.open("Datos.txt");
    prueba p3, p4;
    p3.Leer(fin);
    p4.Leer(fin);
    fin.close();

    p3.prt();
    cout << endl;
    p4.prt();
}
```



Input/output files

The extraction operator ">>", which is used to read from a file, works in the same way as ***cin***, i.e. it only reads words and not phrases.

To read lines from a file, there is another way to do it, using the ***getline()*** function, which also exists in these classes.

The ***getline()*** function reads from file a number of characters specified in the variable ***nCount*** or until it finds the end-of-line character '***\n***'. It returns a NULL when it finds the end of the file. Its structure is:

```
istream& getline(unsigned char* puch,int nCount,char dlm='\n');
```

Example:

```
#include <fstream>
#include <iostream>
using namespace std;

void main()
{
    char frase[81];
    fstream fichero;
    fichero.open("data_file.txt", ios::in|ios::_Nocreate);
    while(fichero.getline(frase,80) != NULL)
        cout << frase;
}
```